FLOWLUNA — CORE AUDIO & VIDEO ENGINE

Audit Technique : Skip OP (Saut d'introduction)

Analyse architecturale, mécanismes de détection en cascade, déclenchement UI et cartographie du code

APPLICATION	DATE D'AUDIT	PÉRIMÈTRE	AUDITEUR
FlowLuna Desktop	18 Octobre 2026	Lecteur Vidéo & Marathon	Antigravity AI Engine

Synthèse exécutive

La fonctionnalité **Skip OP** de FlowLuna intègre une architecture de pointe en **cascade hiérarchique à 6 niveaux**, complétée par un module de **snapping chirurgical aux silences audio** (inspiré d'*intro-skipper*). Elle protège rigoureusement l'expérience utilisateur grâce à des **garde-fous anti-dérive temporelle** (détection de drift à ±4.0s) et interdit formellement le saut automatique aveugle lorsque la confiance est basse ou issue d'heuristiques pures.

1 Niveaux, Modes & Options de Configuration

L'implémentation propose une flexibilité complète via des options persistées en base locale et dans les préférences utilisateur du lecteur.

Matrice des 6 niveaux hiérarchiques de la cascade de détection

NIVEAU	NOM DU PALIER	SOURCE	INDICE CONFIANCE	DESCRIPTION & DÉPENDANCES
Niveau 1	Chapitres intégrés	`chapter`	HIGH	Métadonnées conteneur (MKV/MP4), `textTracks` HTML5 ou extraction `ffprobe -show_chapters`.
Niveau 2	Empreinte acoustique locale	`acoustic`	HIGH (LOCAL)	Corrélation croisée sonore multi-épisodes (FFmpeg 200 Hz PCM, enveloppe RMS 20 Hz, seuil Pearson ≥ 0.70).
Niveau 3	Bases communautaires	`api`	HIGH / MODERATE	API REST AniSkip (résolution MAL ID via AniList) avec repli sur Anime-Skip GraphQL (Client ID dédié).
Niveau 4	Analyse des sous-titres	`subtitle`	HIGH	Détection textuelle des résumés (*Recap*) et teasers (*Preview*) dans les fichiers `.srt`, `.ass`, `.vtt`.
Niveau 4.5	Assistance Sémantique IA	`ai`	HIGH	Inférence LLM (Gemini, OpenAI, Ollama, LM Studio) sur les 250 premiers segments textuels horodatés.
Niveau 5	Estimation Heuristique	`heuristic`	LOW	Fallback mathématique standardisé (début 60s, durée 90s). Interdiction stricte d'auto-skip à l'aveugle.
Finition	Snapping Audio / Silences	`snapped`	CHIRURGICAL	Recalage des bornes sur les silences réels (`silencedetect=-30dB`) et fondus au noir (fenêtre ±3.5s).

Options utilisateur disponibles dans l'interface (Settings)

`enableSkipOpening` (défaut: **true**) : Activation globale de la fonctionnalité. Si faux, désactive toute requête et masque le bouton.

`autoSkipOpening` (défaut: **false**) : Déclenche le saut dès l'entrée dans l'opening sans intervention de l'utilisateur.

`enableSilenceSnapping` (défaut: **true**) : Calage automatique sur les silences audio via FFmpeg pour éviter toute coupure abrupte.

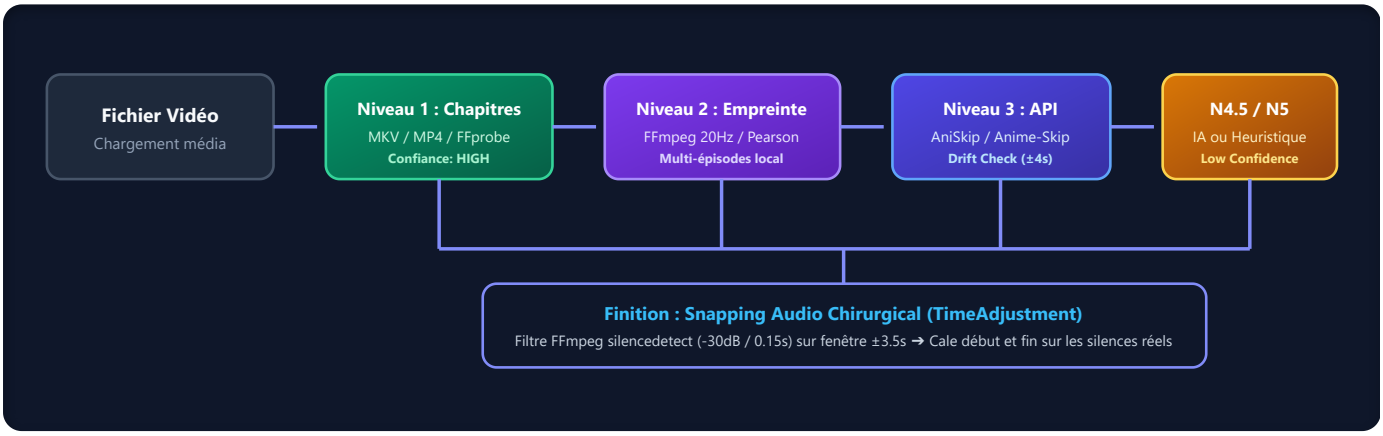
`animeSkipClientId` (optionnel) : Permet de renseigner un Client ID Anime-Skip personnel pour s'affranchir du rate-limiting public.

`skipOpAiMode` / `skipOpAiProvider` : Configuration de l'IA (désactivé, cloud Google/OpenAI ou local Ollama/LM Studio).

`marathonConfig.skipFirstEpisodeOp` (défaut: **false**) : Règle de protection empêchant l'auto-skip sur l'épisode 1 d'une série.

2 Mécanisme & Logique Interne Technique

Le processus de détection est résolu par la fonction maîtresse `resolveVideoMarkersCascade` dès qu'un média vidéo est chargé dans le lecteur.



Détails d'implémentation par niveau

Niveau 1 — Extraction des chapitres vidéo
Vérifie en premier lieu `track.chapters` et les pistes `videoElement.textTracks` (kind *chapters*). Si le conteneur n'est pas encore parsé, appelle l'API locale `/api/video/chapters` qui lance `ffprobe -v quiet -print_format json -show_chapters`. Les titres sont scannés par des Regex tolérantes (anglais, français, japonais) exigeant une durée ≥ 20 secondes.

Niveau 2 — Corrélation acoustique locale (Inspiré d'Intro-Skipper / Plex)
Déclenché via l'endpoint `/api/video/acoustic-match`. Sélectionne un épisode frère dans le même répertoire. FFmpeg extrait 240 secondes en PCM 32-bit float mono à 200 Hz. Le signal est décimé en enveloppe énergétique RMS à 20 Hz, puis comparé par corrélation de Pearson glissante (fenêtre de 90s). Un coefficient **≥ 0.70** garantit l'alignement exact de l'opening sans nécessiter de connexion réseau.

Niveau 3 — API Communautaires & Algorithme de Dérive (Drift Check)
Nettoie le nom de fichier via `parseAnimeTitleAndEpisode` (supprime résolutions, tags `[Release]`, codecs). Résout le MAL ID via AniList GraphQL, puis requête AniSkip. Si absent, requête l'API GraphQL Anime-Skip.
Contrôle de dérive temporelle : Compare la durée retournée par l'API avec la durée réelle du fichier. Si `|referenceDuration - actualDuration| > 4.0s`, `driftDetected` passe à `true` et la confiance est rétrogradée à `moderate`.

Finition — Snapping aux silences audio
Appelle `/api/video/fine-tune-markers` (implémenté en Node.js et en C# .NET). Exécute `ffmpeg -af silencedetect=noise=-30dB:d=0.15` dans une fenêtre de ±3.5s autour du début et de la fin de l'intervalle. Recale au millième de seconde près sur le silence pour éviter les coupures brutes de voix ou d'accords musicaux.

3 Déclenchement & Comportement UI

La surveillance du flux temporel est opérée en temps réel dans le hook `useMarathonController` lors de chaque déclenchement de `timeupdate`.

1. Le Saut Automatique (Auto-Skip)

> **Garde-fous stricts d'Auto-Skip :**
> Pour s'exécuter automatiquement, toutes les conditions suivantes doivent être réunies :
>
> 1. `enableSkipOpening === true` et `autoSkipOpening === true` dans la configuration.
> 2. Épisode 1 protégé si `marathonConfig.skipFirstEpisodeOp === false`.
> 3. **Interdiction absolue si** `source === 'heuristic'`
>   (jamais d'auto-skip sur une estimation !).
> 4. **Interdiction absolue si** `driftDetected === true`
>   (jamais d'auto-skip en cas d'écart de durée).
> 5. Le décodeur vidéo doit être prêt (`readyState >= 3` et pas de `seeking` en cours).

Action exécutée : Seek instantané vers `op.end`. Un toast informatif discret apparaît en bas à droite : `[•] Opening passé automatiquement (+90s)` avec point clignotant cyan pendant 2.5 secondes.

2. Le Saut Manuel avec Bouton Flottant (`SkipOpeningButton`)

Le bouton apparaît dès que la tête de lecture entre dans `[op.start, op.end[` si le saut automatique n'est pas activé **OU si un risque d'incertitude est détecté** (drift temporel ou estimation heuristique).

BADGE AFFICHÉ	COULEUR & STYLE	CONDITION DE DÉCLENCHEMENT	COMPORTEMENT DU BOUTON
CHAPITRE	VERT ÉMERAUDE	`source === 'chapter'`	Bouton standard, masqué si auto-skip actif.
EMPREINTE AUDIO	INDIGO / VIOLET	`source === 'acoustic'`	Bouton standard, masqué si auto-skip actif.
COMMUNAUTAIRE	BLEU CIEL	`source === 'api'` sans drift	Bouton standard, masqué si auto-skip actif.
API (DÉCALAGE)	AMBRE / JAUNE	`driftDetected === true`	**Affichage forcé** même si auto-skip actif (icône alerte).
HEURISTIQUE	ARDOISE / GRIS	`source === 'heuristic'`	**Affichage forcé** avec avertissement d'incertitude.

Ergonomie & Contrôles du Bouton Flottant :

- **Raccourci Clavier Dédié :** La touche `S` active instantanément le saut sans utiliser la souris.
- **Toggle Rapide Auto-Skip :** Une bascule `ToggleRight` / `ToggleLeft` est adossée au bouton pour inverser l'auto-skip à la volée.
- **Disparition Contextuelle :** Masqué instantanément dès que `currentTime >= op.end`.

4 Cartographie Complète des Fichiers Clés

Répertoire exhaustif des composants, services et endpoints à prendre en compte pour une future refonte :

FICHIER	COUCHE	FONCTIONS / ÉLÉMENTS CLÉS	RÔLE & RESPONSABILITÉ
`src/services/openingDetectorService.ts`	Service Client	`resolveVideoMarkersCascade()` `resolveChaptersMarkers()` `resolveAniskipMarkers()` `fineTuneIntervalMarkers()`	Moteur principal de résolution en cascade, gestion du cache local, parser de titres et requêtes API.
`src/hooks/useMarathonController.ts`	Hook React	`useMarathonController()` `skipOpening()` Surveillance `currentTime`	Orchestration de la lecture en continu, application des règles d'auto-skip, interaction avec l'ending et le marathon.
`src/hooks/useOpeningDetector.ts`	Hook React	`useOpeningDetector()`	Contrôleur unitaire simplifié pour le saut d'opening isolé et le stockage local du toggle.
`src/components/SkipOpeningButton.tsx`	Composant UI	`SkipOpeningButton` `getSourceBadge()`	Rendu visuel du bouton flottant, badges d'état/confiance, raccourci `S` et toggle auto-skip.
`src/components/VideoPlayer.tsx`	Composant UI	Intégration du hook Capture touche clavier `S`	Injecte les marqueurs dans le player vidéo HTML5 et monte le composant `SkipOpeningButton`.
`src/components/SettingsModal.tsx`	Composant UI	Panneau Réglages Vidéo	Configuration utilisateur (toggles, clé Anime-Skip, modèles IA Cloud/Local, mode marathon).
`server.ts`	Backend Node.js	`/api/video/chapters` `/api/video/acoustic-match` `/api/video/fine-tune-markers`	Exécution locale de FFprobe (chapitres) et FFmpeg (corrélation acoustique et détection de silences).
`FlowLuna.Windows/.../IntroSkipperService.cs`	Backend .NET C#	`FineTuneIntervalAsync()` `MatchAudioOpeningAsync()`	Version C# compilée native haute performance pour l'exécutable Windows autonome de FlowLuna.
`src/types.ts`	Modèles TypeScript	`SkipInterval`, `SkipSource` `VideoSkipMarkers`, `MarathonConfig`	Définition des interfaces TypeScript, types de confiance, sources et structures de données.

5 Recommandations pour la Future Refonte

1. **Unification des hooks :** Fusionner `useOpeningDetector` et `useMarathonController` pour éviter toute redondance de logique ou de timers de toasts.
2. **Pré-calcul en tâche de fond (Background Worker) :** Déporter l'analyse acoustique (Niveau 2) et le snapping lors de l'indexation de la bibliothèque vidéo au lieu de l'exécuter à la volée au chargement du fichier.
3. **Gestion fine des génériques multiples :** Prendre en compte les animes disposant de plusieurs openings au sein d'une même saison (changement à l'épisode 13 par exemple).
4. **Interface visuelle sur la timeline :** Dessiner visuellement la zone de l'opening (bande colorée ou marqueur de chapitre) directement sur la barre de progression (seek bar) du lecteur.